Student 21 **Lakshmi Narayanan E** 192524249RD 5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.util.Scanner;
class ATM{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        try{
            int balance = sc.nextInt();
            int amount = sc.nextInt();
            if(amount<=0){
                System.out.println("Invalid withdrawl amount");
            }
            else if(amount>balance){
                System.out.println("Insufficient balance");
            }
            else{
                int remainaing=balance/amount;
                System.out.println("With draw Successful");
                System.out.println("Remaining balace = "+(balance-amount));
            }
        }
        catch(Exception e){
            System.out.println("Invalid Input");
        }
        finally{
            System.out.println("Transaction completed");
        }
    }
}
```

INPUT

```
50000
1000
```

OUTPUT

```
With draw Successful
Remaining balace = 49000
Transaction completed
```

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.Scanner;
class product{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        String [] products = {"Laptop","Mobile","Tablet","HeadPhone,Keyboard"};
        int productNumber = sc.nextInt();
        System.out.println("Selected producted : "+ products[(productNumber-1)]);
        System.out.println("Search completed.......");
    }
}
```

INPUT

```
3
```

OUTPUT

```
Selected producted : Tablet
Search completed.......
```

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.util.Scanner;
class productsearch{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String [] products ={"laptop","Mobile","Tablet","HeadPhones","Keyboard"};
        int productNumber=sc.nextInt();
        System.out.println("Selected Product : "+ products[(productNumber-1)]);
        System.out.println("Search completed........");
    }
}
```

INPUT

```
2
```

OUTPUT

```
Selected Product : Mobile
Search completed........
```

Q4. Railway Reservation – Synchronization Debugging

CODE

```
class RailwayReservation{
    public static void main(String [] args){
        Reservation reservation = new Reservation();
        Customer c1 = new Customer(reservation,"customer 1");
        Customer c2 = new Customer(reservation,"customer 2");
        c1.start();
        c2.start();
    }
}
class Reservation{
    int seats= 1;
    synchronized void bookSeat(String customer){
        if(seats>0){
            System.out.println(customer +"is booking a seat");
            try{
                Thread.sleep(1000);
            }
            catch(InterruptedException e){
                System.out.println("Thread Interrupted");
            }
            seats--;
            System.out.println(customer+"Booking Successful");
        }
        else{
            System.out.println(customer+"No seat available");
        }
    }
}
class Customer extends Thread{
    Reservation reservation;
    String customerName;
    Customer(Reservation reservation,String customerName){
        this.reservation = reservation;
        this.customerName = customerName;
    }
    public void run(){
        reservation.bookSeat(customerName);
    }
}
```

OUTPUT

```
customer 1is booking a seat
customer 1Booking Successful
customer 2No seat available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
import java.util.Scanner;
class BankAccount{
    int balance = 10000;
    synchronized void withdraw(int amount){
        if(balance>=amount){
            System.out.println(Thread.currentThread().getName()+"Withdrawing"+amount);

            try{
                Thread.sleep(100);
            }
            catch(InterruptedException e){
                System.out.println("Interrupted");
            }
            balance = balance - amount;
            System.out.println(Thread.currentThread().getName()+"Withdrawl Successful");
        }
        else{
            System.out.println(Thread.currentThread().getName()+"Insufficeint balance");
        }
    }
    public static void main(String args[]){
        BankAccount account = new BankAccount();
        ATM atm1 = new ATM(account,7000);
        ATM atm2 = new ATM(account,6000);
        atm1.setName("ATM-1");
        atm2.setName("ATM-2");
        atm1.start();
        atm2.start();
        try{
            atm1.join();
            atm2.join();
        }
        catch(InterruptedException e){
            System.out.println("Interrupted");
        }
        System.out.println("Final Balance = "+account.balance);
    }
}
class ATM extends Thread{
    BankAccount account;
    int amount;
    ATM(BankAccount account,int amount){
        this.account = account;
        this.amount = amount;
    }
    public void run(){
        account.withdraw(amount);
    }
}
```

OUTPUT

```
ATM-1Withdrawing7000
ATM-1Withdrawl Successful
ATM-2Insufficeint balance
Final Balance = 3000
```